

- ① - merge 2 SSL.
- end $\rightarrow$ only resulting list should $\exists$. (at. extra space)
 - reprs. of SSL
 - implem + complexity.

(see PSA row 6 u for K lists)
 $\hookrightarrow$ but that uses extra space.
 $\hookrightarrow$ a heap.

SSLNode

info: TComp
 next: $\uparrow$ SSLNode

SSL

head: $\uparrow$ SSLNode
 rel: $\uparrow$ relation

function merge (l_1, l_2) is

$p_1 \leftarrow l_1.\text{head}$
 $p_2 \leftarrow l_2.\text{head}$
 $\text{result.head} \leftarrow \text{Nil}$
 $\text{last} \leftarrow \text{Nil}$
 while $p_1 \neq \text{Nil}$ and $p_2 \neq \text{Nil}$ ex

if $[p_1].\text{info} \leq [p_2].\text{info}$ then
 selected $\leftarrow p_1$
 $p_1 \leftarrow [p_1].\text{next}$
 else
 selected $\leftarrow p_2$
 $p_2 \leftarrow [p_2].\text{next}$

if $\text{result.head} = \text{Nil}$ then
 $\text{result.head} \leftarrow \text{selected}$
 else
 $[\text{last}].\text{next} \leftarrow \text{selected}$

$\Theta(m_1 + m_2)$

$\Theta(1)$ e.s.

if $p_1 \neq \text{Nil}$ then
 $\text{rest} \leftarrow p_1$
 else
 $\text{rest} \leftarrow p_2$

$\leftarrow$ remaining chunk (take it all at once)

if $\text{result.head} = \text{Nil}$ then
 $\text{result.head} \leftarrow \text{rest}$
 else
 $[\text{last}].\text{next} \leftarrow \text{rest}$

var.head vs $///$
 $[\text{var}].\text{next} \dots$

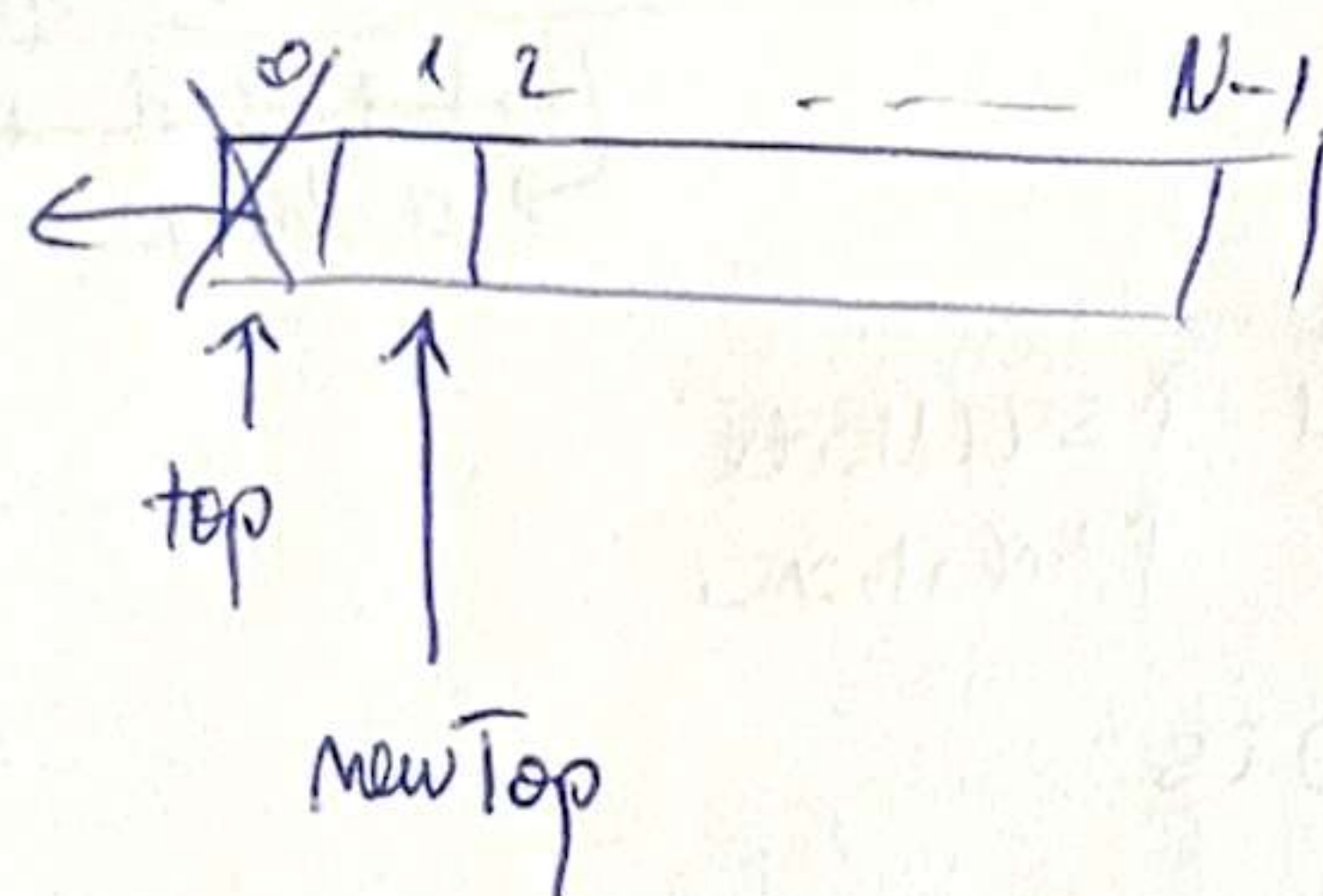
$\hookrightarrow$ go in memory at that addr.

$l_1.\text{head} \leftarrow \text{Nil}$
 $l_2.\text{head} \leftarrow \text{Nil}$
 $\text{merge} \leftarrow \text{result}$

2. 1) tightest bound, Big-O.

a) pop from a stack implem. on an array containing N elems.

WC: $O(1)$ → remove top elem. by changing top index.



b) find in a HT, collis. res. w open addr. (linear probing), containing N elems; $N^2 = \text{table size}$; $h(k, i) = (h'(k) + i) \% m$

WC: $O(N^2)$ → search may inspect whole table.

c) insert in HT, collis. res w. sep. chaining, containing N elems, $\text{table size} = N^2$.

WC: $O(m)$ → all keys hash to the same pos, making a single chain of N elems; even though insertion cost is $O(1)$, we need to traverse the whole list to check duplicates before inserting.

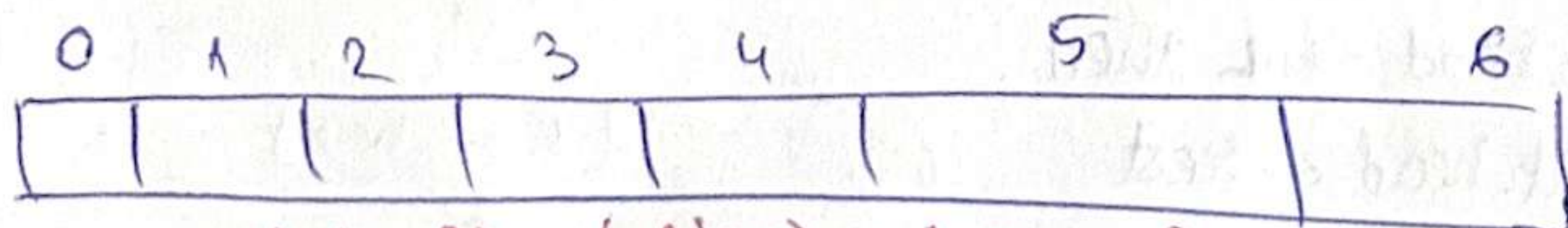
$$d) T(m) = \log_2(\log_2(N+N)) + (\log_2 N)^2$$

⇒ $O((\log_2 N)^2)$
"tightest bound",

leading term.

even though $O((\log_2 N)^{100})$ could also be valid.

2) HT, $m = 7$ pos., coll. res., open addressing, quadratic probing with $c_1 = 3$, $c_2 = 2$, div. method, probe seq. for 44 - good hash fct?



$$h(k, i) = (h'(k) + i \cdot c_1 + i^2 \cdot c_2) \% m$$

$$h'(k) = 44 \% 7 = 2$$

$$h(44, 0) = (2 + 0) \% 7 = 2$$

$$h(44, 1) = (2 + 1 \cdot 3 + 1^2 \cdot 2) \% 7 = 7 \% 7 = 0$$

$$h(44, 2) = (2 + 2 \cdot 3 + 2^2 \cdot 2) \% 7 = 16 \% 7 = 2$$

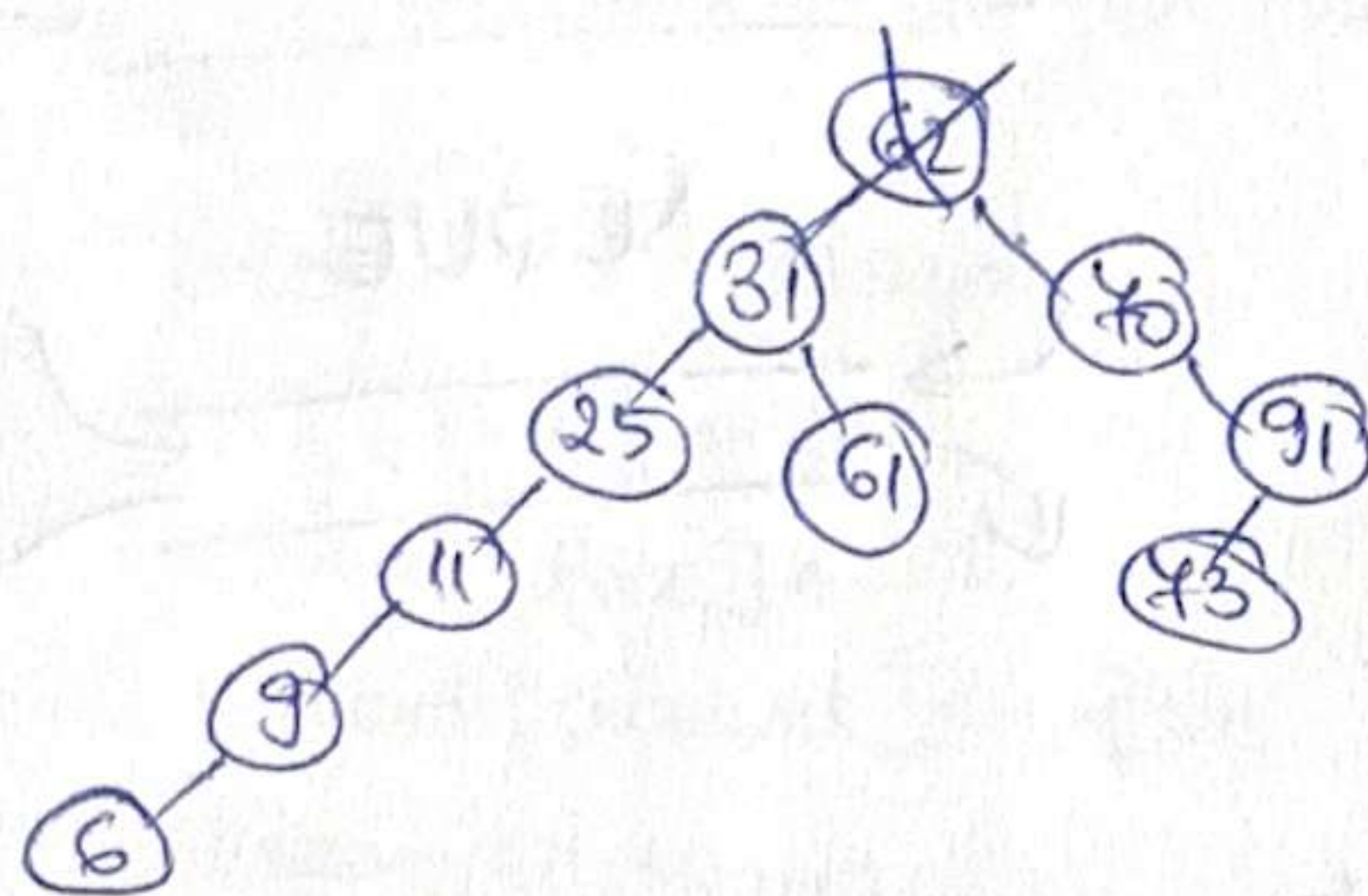
$$h(44, 3) = (2 + 3 \cdot 3 + 3^2 \cdot 2) \% 7 = 19 \% 7 = 5$$

$$h(44, 4) = 4$$

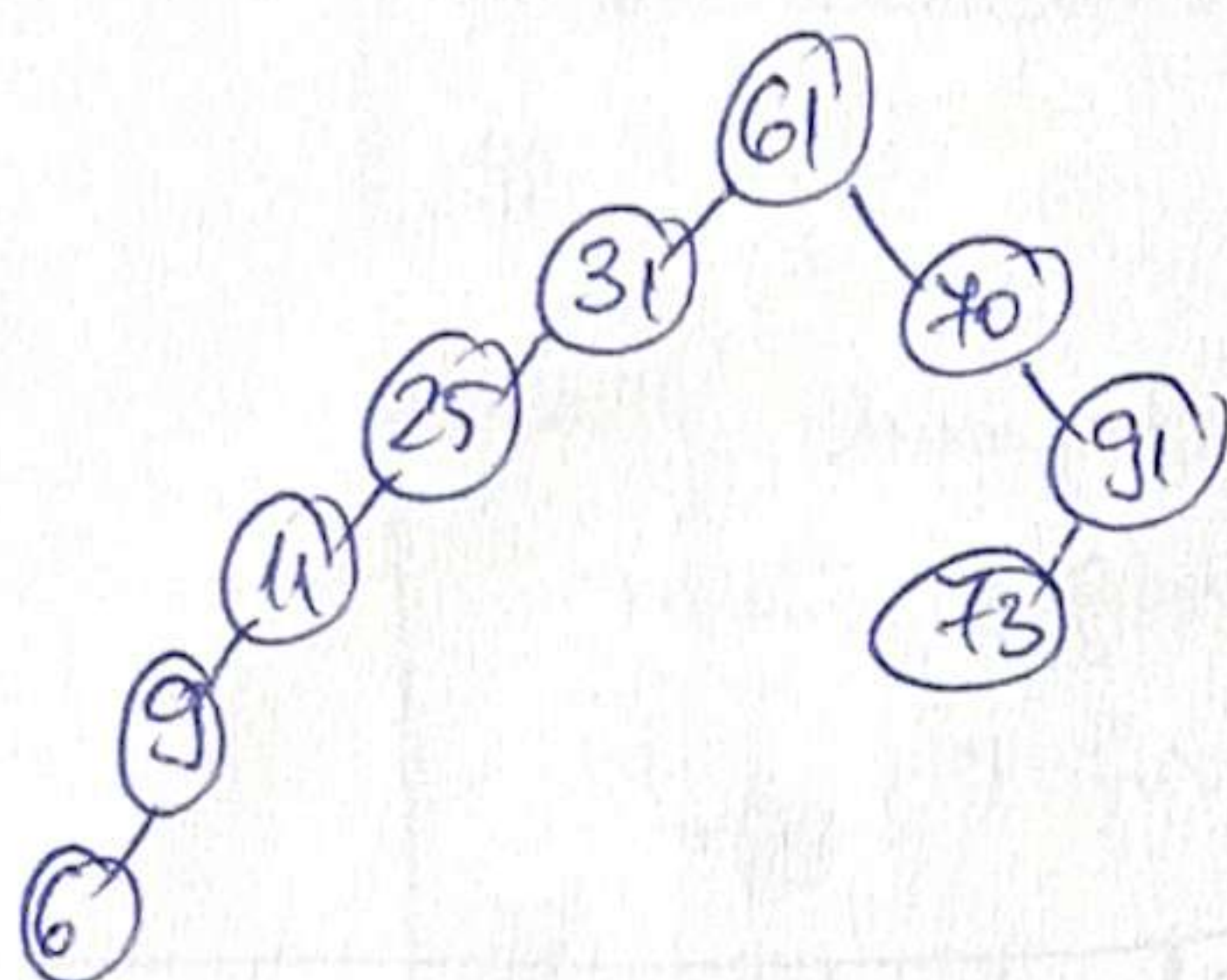
$$h(44, 5) = 4; h(44, 6) = 1$$

⇒ probe sequence: 2, 0, 2, 1, 4, 4, 1
⇒ NOT A good hash fct.
because it does not visit all table positions and there are many collisions

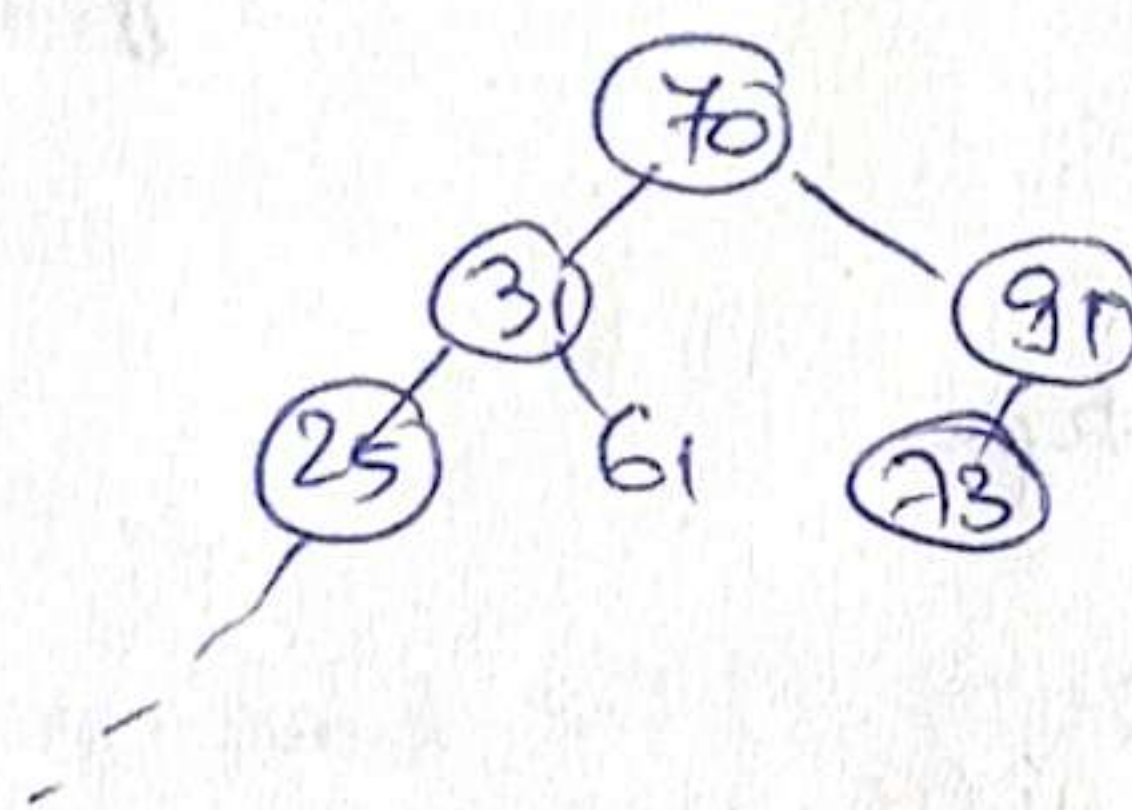
3) BST. insert: 62, 31, 70, 91, 25, 11, 9, 61, 73, 6.



I. predecessor: 61



II. Successor: 70



3. 1. $N^2 + N^2 \log_2 N$

a) $\Omega(N^2)$

~~b) $O(N^2)$~~

c)

d) $O(N^3)$.

2. Stack $\rightarrow$ array implem.
which op $\rightarrow$ lim. time $\rightarrow$ wc?

a. $\Theta(1)$ pop

b. push when $nrElem < cap.$ $\rightarrow \Theta(1)$ (no resize)

c. isEmpty $\rightarrow$ check $top = 0 \rightarrow \Theta(1)$

d. top $\rightarrow \Theta(1)$

(e) None.

3. Best repres. for a Priority Queue $\rightarrow$ push/pop in $O(\log_2 m)$ wc
 $\rightarrow$ top in $\Theta(1)$

a) BST $\rightarrow$ for priority I would need them sorted $\Rightarrow$ skew tree $\Rightarrow O(N)$.

b) OA $\rightarrow$ keep array sorted $\rightarrow$ requires shifting (top) $\Rightarrow O(N)$

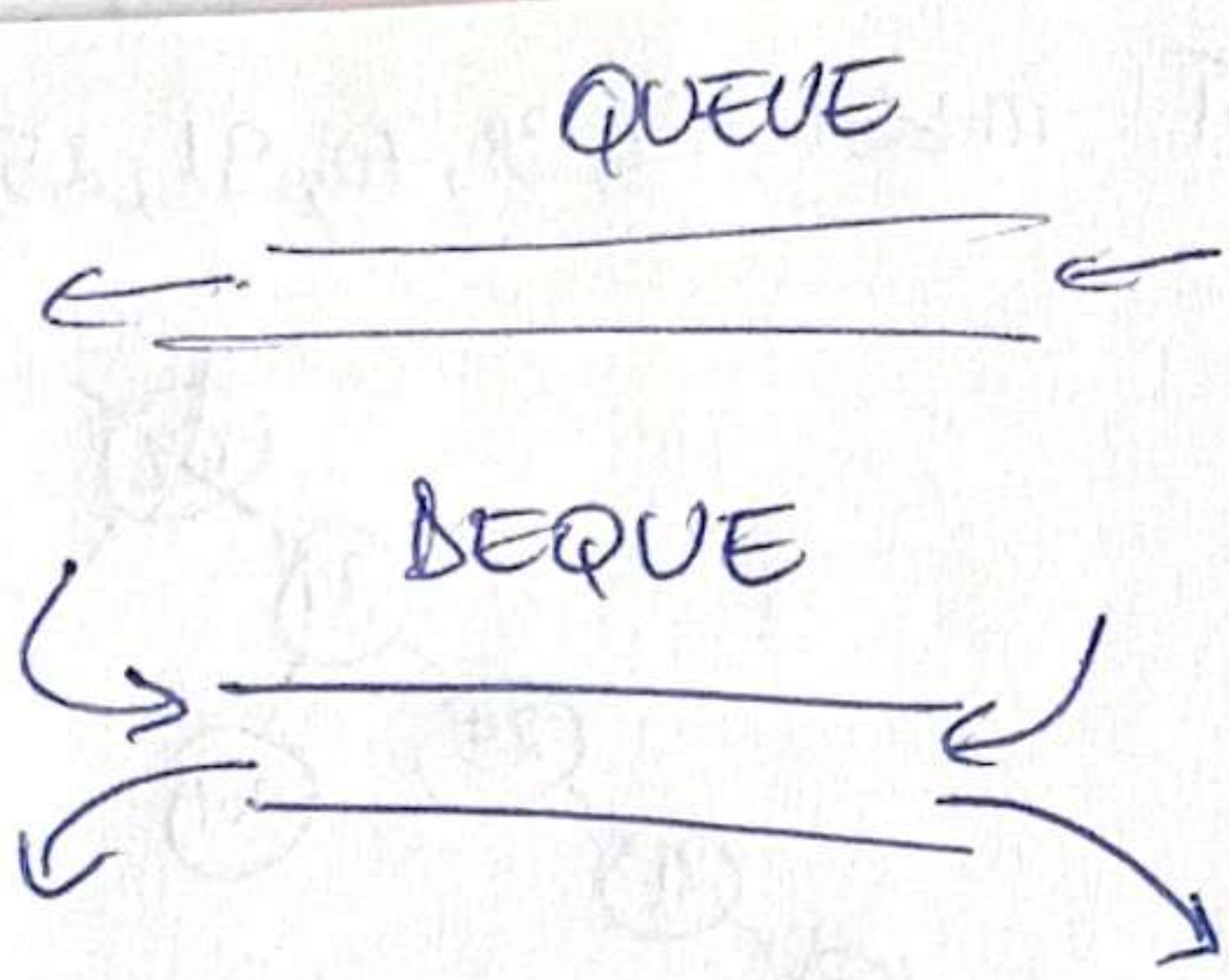
[c] Binary Heap $\rightarrow$ BT mapped into an array $\rightarrow$ wc: bubble-down

d) AL $\rightarrow$ min/max $\rightarrow$ top in $O(\log_2 N)$, push/pop in $O(\log_2 N)$ ✓

5) ~~dequeue~~ → queue

only these op:

- (d) push-back
- pop-front



6) Perfect hashing restriction:

- (c) all elems known.

4. Set → HT, coalesced chaining

- set repres.
- insert
- complexity

Set

↳ no pos, no order, unique;

coalesced:

HT:

- T: Tkey[] ✓
- next: Integer[] ✓
- me: Integer ✓
- firstEmpty: Integer ✓
- h: Tfunction ✓
- nrElems: Int

pos	0	1	2	...
T	-1	-1	-1	...
next	-1	-1	-1	...

characteristic to a set.

next and T are both init with -1 at the beginning

subalgorithm insert(s, e) is

if search(s, e) then
@throw ---

← already exists! (A set does not allow duplicates).

if s.firstEmpty = s.me then
@resize and rehash

← reached "capacity".

pos ← s.h(e)

if s.T[pos] = -1 then { empty, we can put elem. there

s.T[pos] ← e

s.next[pos] ← -1

if pos = s.firstEmpty then
changeFirstEmpty(s)

else

current ← pos


```

while s.next[current]  $\neq$  -1 then ex
  current  $\leftarrow$  s.next[current]
  free  $\leftarrow$  s.firstEmpty
  s.T[free]  $\leftarrow$  e
  s.next[free]  $\leftarrow$  -1
  s.next[current]  $\leftarrow$  free
  changeFirstEmpty(s)
  s.nElements  $\leftarrow$  s.nElements + 1

```

subalg changeFirstEmpty(s) is

```

while s.firstEmpty < s.m and s.T[s.firstEmpty]  $\neq$  -1 execute
  s.firstEmpty  $\leftarrow$  s.firstEmpty + 1

```

BC: $\Theta(1)$

WC: $\Theta(n)$

T: $\mathcal{O}(n)$

- 5) $\rightarrow$ param: postorder, inorder (arrays $\rightarrow$ access based on pos.)
 $\rightarrow$ build tree
 $\rightarrow$ BT repres.
 $\rightarrow$ complexity

postorder: left Right Root
 $\downarrow$
 last elem in postorder = ROOT $\uparrow$

Node:

info: TElem

left: $\uparrow$ Node

right: $\uparrow$ Node

BT:

root: $\uparrow$ Node

inorder: left of ROOT = left sub
 right -- = right subtree

RECURSIVE $\rightarrow$ NEEDS HELPER FCT! $\downarrow$

~~function buildTree~~

subalgorithm buildTree(inorder, postorder, n) is

```

tree root
  bt.root  $\leftarrow$  build(inorder, 0, n-1, postorder, 0, n-1)

```


function build(inorder, inStart, inEnd, postorder, postStart, postEnd) is

if inStart > inEnd then
 build ← Nil
 return

BASE CASE

(range of indices is empty)

rootValue ← postorder[postEnd] // last elem. in postorder = ROOT
root ← allocate

[root].info ← rootValue

[root].left ← Nil

[root].right ← Nil

post++
inorder: L Root R

pos ← inStart

while inorder[pos] ≠ rootValue execute

 pos ← pos + 1

decrement back? root is on [pos]?

leftSize ← pos - inStart

[root].left ← build(inorder, inStart, pos - 1, postorder, postStart, postStart + leftSize - 1)

[root].right ← build(inorder, pos + 1, inEnd, postorder, postStart + leftSize, postEnd - 1)

build ← root

WC: $\Theta(n^2)$ → for each node → search linearly in inorder.